

Software Testing

Not from your notes — drafted fresh. This is one of the four gap topics you asked for: your SET notes don't cover testing at all, but it's one of the most reliably-asked SE topics in SDE interviews, so this draft builds it from scratch.

1. What Testing Actually Is

Software testing is the process of executing a program (or part of one) with the specific intent of finding defects in it. The famous Dijkstra line is worth internalizing for interviews: *"Testing shows the presence of bugs, not their absence."* No amount of testing can prove a nontrivial program is defect-free — it can only increase confidence by failing to find defects despite trying hard to.

Testing's job is **detecting** defects, not fixing them — fixing a detected defect is debugging, a separate activity (this distinction, and how it relates to verification/validation and QA, is covered in the Software Quality topic).

2. Testing Approaches: White-Box, Black-Box, and Gray-Box

	White-Box (Structural)	Black-Box (Functional)	Gray-Box
What's examined	The internal code/logic itself	Only inputs and outputs — internal logic is opaque	A mix — some internal knowledge, tested mostly externally
Who typically does it	Developers	Testers / QA engineers	Testers with partial design knowledge
Goal	Exercise every path, branch, or statement in the code	Verify the software meets its functional requirements	Combine coverage insight with functional focus
Typical level	Unit testing	System / acceptance testing	Integration testing

White-Box Techniques

These are about **coverage** — how much of the code's actual structure has been exercised:

- **Statement coverage** — has every line of code executed at least once?
- **Branch/Decision coverage** — has every branch (both the true and false outcome of every decision) been exercised at least once?

- **Path coverage** — has every independent path through the code been exercised? This is exactly what **Cyclomatic Complexity** (covered in the Metrics & Cost Estimation topic) quantifies — the cyclomatic complexity of a function is the *minimum number of test cases* needed to achieve full path coverage of it.

Black-Box Techniques

Equivalence Partitioning — divide the input domain into partitions where the software is expected to behave the same way for any input in that partition, then test just one representative value from each partition (instead of exhaustively testing every possible input).

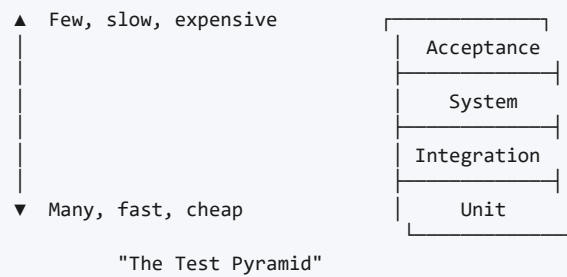
Boundary Value Analysis (BVA) — defects cluster disproportionately at the *edges* of valid input ranges, not in the middle, so BVA specifically tests values at and just around each boundary.

Worked example: a form field accepts an age between 18 and 60 (inclusive) for eligibility.

- **Equivalence partitions:**
 - Invalid (too low): $\text{age} < 18$
 - Valid: $18 \leq \text{age} \leq 60$
 - Invalid (too high): $\text{age} > 60$
- → Equivalence partitioning needs just **3 test cases**, e.g., `age = 10` (invalid-low), `age = 35` (valid), `age = 75` (invalid-high).
- **Boundary values:** the interesting values are right at each edge:
 - `age = 17` (just below the lower boundary — should be rejected)
 - `age = 18` (exactly on the lower boundary — should be accepted)
 - `age = 60` (exactly on the upper boundary — should be accepted)
 - `age = 61` (just above the upper boundary — should be rejected)
- → BVA needs **4 test cases**, specifically targeting the off-by-one bugs (`<` vs `<=` , `>` vs `>=`) that equivalence partitioning's single "valid" representative (`age = 35`) would never catch.

Interview soundbite: "Equivalence partitioning tells you *how many* regions of input space matter; boundary value analysis tells you *where within those regions* bugs actually hide. A test suite using only equivalence partitioning would completely miss an off-by-one error at `age = 18` , since `age = 35` passes either way."

3. Levels of Testing



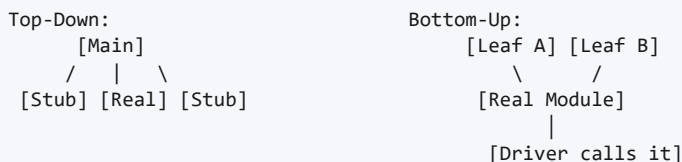
Level	Scope	Who writes it	Finds
Unit Testing	A single function/method/class in isolation	Developers	Logic errors within one unit
Integration Testing	Interactions between two or more units/modules	Developers / QA	Interface mismatches, wrong assumptions about how modules cooperate
System Testing	The complete, integrated system, end to end	QA team	Whether the whole system meets its specified requirements
Acceptance Testing	The system against the <i>customer's</i> actual needs	Customer / end users (or QA on their behalf)	Whether this is actually the software the customer wanted

The **Test Pyramid** shape is itself a piece of advice, not just a description: it argues you should have *many* fast, cheap unit tests, a moderate number of integration tests, and comparatively *few* slow, expensive, end-to-end/system tests — inverting the pyramid (few unit tests, heavy reliance on slow UI-driven system tests) is a well-known anti-pattern because it makes the test suite slow to run and failures hard to localize (a failing end-to-end test tells you *something* is broken somewhere across the whole system, not *what*).

4. Integration Testing Strategies

When combining units into a working whole, four common strategies exist:

Strategy	Approach	Needs
Big Bang	Integrate <i>all</i> modules at once, then test the whole thing together	Nothing extra — but defects are hard to isolate ("something broke somewhere among 40 modules")
Top-Down	Start with the top-level module, integrate downward, one level at a time	Stubs — dummy placeholder implementations of not-yet-integrated <i>lower</i> modules
Bottom-Up	Start with the lowest-level modules, integrate upward	Drivers — dummy calling code that invokes the not-yet-integrated <i>higher</i> modules being tested
Sandwich / Hybrid	Top-down and bottom-up simultaneously, meeting in the middle	Both stubs and drivers



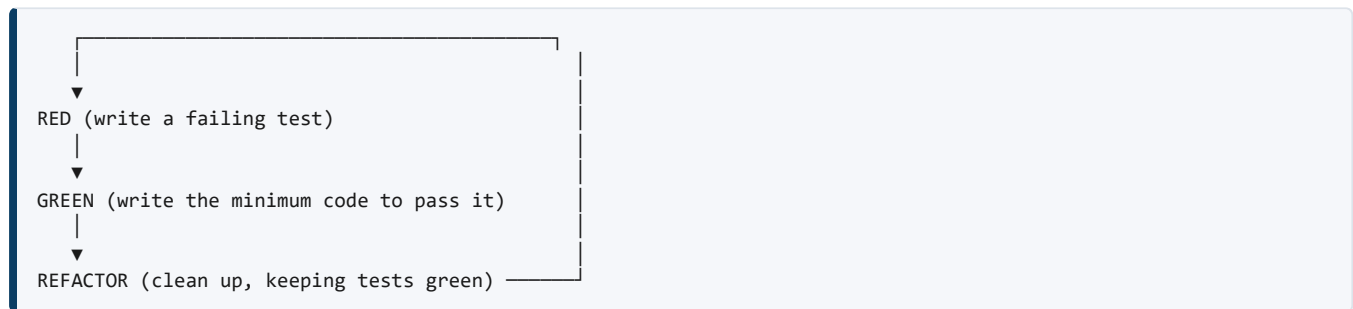
Stub vs. Driver, one line: a **stub** is a fake *callee* (stands in for a module not yet built, so the module being tested has something to call); a **driver** is a fake *caller* (stands in for the not-yet-built code that would normally call the module being tested).

5. Other Common Testing Types

- **Regression Testing** — re-running previously-passing tests after a change, to confirm the change didn't break something that used to work. This is exactly what makes refactoring (from the Design Concepts topic) safe to do in practice — without a regression suite, "improve the internals without changing external behavior" is just a hope, not something you can verify.
- **Smoke Testing / Sanity Testing** — a quick, shallow pass confirming the most critical functionality works at all, before investing time in deeper testing (named after the hardware-testing idea of "turn it on and see if it smokes").
- **Alpha vs. Beta Testing** — Alpha testing happens in-house, at the developer's own site, typically by internal staff acting as surrogate users; Beta testing happens at one or more actual customer sites, with real users, in a real (or close to real) environment, before general release.
- **Performance / Load / Stress Testing** — performance testing measures response time/throughput under expected conditions; load testing pushes toward expected peak usage; stress testing pushes *past* expected limits to see how/where the system breaks.

6. Test-Driven Development (TDD)

TDD inverts the usual order: **write the test before writing the code that makes it pass.**



1. **Red** — write a test for a feature that doesn't exist yet; it fails (there's nothing to make it pass).
2. **Green** — write the simplest possible code that makes the test pass, without worrying yet about elegance.
3. **Refactor** — now that the behavior is locked in by a passing test, clean up the implementation, relying on the test to catch any regression introduced during cleanup.

Benefits: forces requirements to be expressed as concrete, checkable examples before coding starts; produces a regression suite as a natural byproduct of development rather than an afterthought; tends to push toward more testable (hence more decoupled, per the coupling/cohesion "golden rule") designs, since code that's hard to test in isolation is usually code with too many dependencies. TDD is one of the core practices of **Extreme Programming (XP)** — covered in the Agile Methodologies topic.

Interview Questions With Answers

Q1. Dijkstra said "testing shows the presence of bugs, not their absence." What does this mean practically for how much testing is "enough"?

Answer: This means that passing a test suite only tells you the specific scenarios that suite covers behave correctly — it says nothing about the infinite number of scenarios the suite didn't check, so there's no test suite size at which you can definitively declare a nontrivial program "proven correct" through testing alone. Practically, this reframes "how much testing is enough" away from an unanswerable absolute question toward a risk-management question: enough testing means enough coverage of the scenarios most likely to occur and most costly if wrong (using techniques like equivalence partitioning and boundary value analysis to make that coverage systematic rather than arbitrary), combined with a fast, cheap regression suite that keeps confidence high as the code continues to change — not an attempt at some impossible complete verification through testing alone.

Q2. Explain why Boundary Value Analysis catches bugs that Equivalence Partitioning, used alone, would miss.

Answer: Equivalence Partitioning assumes that any input within a given partition will be treated identically by the software, so testing one representative value from each partition should be sufficient — but this assumption specifically breaks down exactly at the *edges* between partitions, which is where off-by-one errors (using `<` instead of `<=`, or vice versa) actually live in real code. If a valid range is meant to be 18–60 inclusive but the code mistakenly uses `age > 18` instead of `age >= 18`, testing the equivalence-partition representative `age = 35` would still pass, completely hiding the bug — only a test at `age = 18` itself would reveal that the boundary is being handled incorrectly. This is exactly why BVA is used *alongside* equivalence partitioning rather than as a replacement for it: partitioning tells you which regions of the input space to sample from, and BVA tells you that the boundaries of those regions need dedicated attention that an arbitrary representative value from the middle of the partition won't provide.

Q3. What's the difference between a stub and a driver in integration testing, and why does Top-Down integration need stubs specifically, rather than drivers?

Answer: A stub is a fake, simplified implementation standing in for a module that hasn't been integrated yet, used when the module *being tested* needs to call something below it that doesn't exist yet; a driver is fake calling code standing in for a module that hasn't been integrated yet, used when the module being tested needs to be called by something above it that doesn't exist yet. Top-Down integration starts with the highest-level module and works downward, meaning at any point during the process, the modules *above* the current level are already integrated and real, but the modules *below* the current level haven't been integrated yet — so the current module needs something to call in their place, which is exactly what a stub provides; there's no need for a driver in this direction, since the real caller (the already-integrated module above) already exists.

Q4. A team has excellent unit test coverage (95%+) but their system frequently breaks in production due to two services disagreeing about the exact shape of the data they exchange. What testing gap does this point to, and why wouldn't more unit tests fix it?

Answer: This points to a gap in **integration testing** specifically — unit tests, by design, test each service in isolation, typically using mocked or stubbed versions of the other service rather than the real thing, so a unit test suite can pass with 100% coverage while still being built on an incorrect assumption about what the other service actually sends or expects. Adding more unit tests wouldn't fix this because the defect isn't inside either service's internal logic — each service may be functioning perfectly correctly according to its own internal understanding of the contract — the defect is specifically in the *agreement between* the two services, which by definition can only be caught by a test that exercises both services together, i.e., an integration test (or a contract test verifying both sides against a shared, explicit interface specification).

Q5. Explain the "inverted test pyramid" anti-pattern and why it's considered a problem even if it achieves the same overall test coverage as a normal pyramid.

Answer: An inverted test pyramid has relatively few fast unit tests and a heavy reliance on slow, expensive end-to-end/system tests to catch most defects — the opposite of the recommended shape (many unit tests, fewer integration tests, fewest end-to-end tests). Even at equivalent overall coverage, this is a problem for two practical reasons: speed and diagnosability. End-to-end tests are inherently slow (often requiring a full running system, real or simulated external dependencies, and UI interaction), so a suite dominated by them takes much longer to run, slowing down the developer feedback loop that makes frequent testing useful in the first place. And when an end-to-end test fails, it typically only tells you *that* something is broken somewhere across the entire system under test, not *which* specific unit or interaction is at fault — whereas a failing unit test points precisely at the function or class responsible, making the inverted pyramid's failures both slower to discover and slower to diagnose once discovered.

Q6. What is the Red-Green-Refactor cycle in TDD, and why is writing the test *before* the code considered valuable rather than just an arbitrary ordering preference?

Answer: Red-Green-Refactor is TDD's core loop: first write a test for behavior that doesn't exist yet, which fails since there's nothing to satisfy it (Red); then write the minimum code needed to make that test pass, without worrying about elegance yet (Green); then clean up the implementation while relying on the now-passing test to immediately flag any regression introduced during cleanup (Refactor). Writing the test first is valuable, rather than an arbitrary ordering choice, because it forces a concrete, checkable definition of "done" to exist *before* any implementation bias can creep in — if the test is written after the code, there's a natural (even if unconscious) tendency to write a test that confirms what the code already does, rather than a test that genuinely specifies what the code is *supposed* to do; writing the test first also guarantees that every line of production code that gets written has an associated failing test that motivated it, which is what makes the resulting test suite an actual specification of behavior rather than an afterthought bolted onto already-written code.

Q7. How does Cyclomatic Complexity (from the Metrics topic) connect directly to white-box path-coverage testing?

Answer: Cyclomatic Complexity is literally defined as the number of independent paths through a piece of code's control flow graph, and achieving full **path coverage** in white-box testing means writing at least one test case for every one of those independent paths — so Cyclomatic Complexity is not just correlated with testing effort, it *is* the minimum number of test cases required to achieve full path coverage of that code. This gives white-box testing a very concrete, actionable target: rather than vaguely aiming to "test thoroughly," a tester can compute a function's cyclomatic complexity and know precisely how many test cases, hitting which specific decision-outcome combinations, are needed to guarantee every independent path has been exercised at least once — and a function with unusually high cyclomatic complexity is both harder to achieve full path coverage on and, per the Metrics topic, empirically more likely to harbor defects, making it a natural priority target for testing effort.

Q8. Scenario: A payment feature passes all its unit tests and all its integration tests, but a customer reports that duplicate charges started appearing after the last deployment, only on the checkout page and only for users on a slow network connection. Which testing type(s) would most likely have caught this before release, and why did unit and integration testing miss it?

Answer: This is most likely a defect that would be caught by **system testing** (specifically testing the full, integrated application under realistic conditions, including the checkout page's actual UI behavior) combined with a form of **performance/load-adjacent testing** simulating slow network conditions — the bug pattern (duplicate charges specifically under slow network) strongly suggests a double-submit issue, e.g., a user clicking "Pay" twice because the button doesn't disable itself while waiting for a slow response, or a retry mechanism resubmitting a request that actually succeeded but was slow to acknowledge. Unit tests missed it because they test the payment logic in isolation from network timing and UI behavior entirely — the payment-processing function itself may be perfectly correct given a single, well-formed request. Integration tests missed it because they typically test module interactions with fast, mocked dependencies and don't simulate real-world network latency or literal repeated user interaction with a UI element — neither unit nor typical integration tests are designed to exercise the specific combination of "slow response" and "user impatience/retry behavior" that only shows up when the whole system is exercised end-to-end under realistic, degraded network conditions.